

AM5011 Virtual Reality Engineering

Assignment 2: Dual Quaternions for VR Poses and Blending

Your Name | Roll Number: XXXXXXXX | Unity Version: 2022.3.52f1

Part A — Math and Representation

1. Constructing the unit dual quaternion

Let q_r be a unit rotation quaternion ($\|q_r\| = 1$) and let $t = (t_x, t_y, t_z)$ be a translation vector, embedded as the pure quaternion $T = (t_x, t_y, t_z, 0)$.

A pure rotation has zero dual part:

$$\hat{R} = q_r + \epsilon \cdot 0.$$

A pure translation is represented as

$$\hat{T} = 1 + \frac{1}{2}\epsilon T.$$

Applying the rotation first and the translation second corresponds to $\hat{q} = \hat{T}\hat{R}$:

$$\hat{q} = \left(1 + \frac{1}{2}\epsilon T\right) (q_r + 0 \cdot \epsilon) = q_r + \epsilon \left(\frac{1}{2}Tq_r\right).$$

Derived unit dual quaternion:

$$\hat{q} = q_r + \epsilon q_d, \quad q_d = \frac{1}{2}Tq_r.$$

Since $\|q_r\| = 1$ and $q_r \cdot q_d = \text{Re}(q_d q_r^*) = \text{Re}(\frac{1}{2}T) = 0$, $\hat{q}$ satisfies both unit dual quaternion constraints and validly encodes the 6-DOF pose (R, t) .

2. Recovering R and t from $\hat{q}$

The real part directly gives the rotation, renormalized to remove drift:

$$R = \frac{q_r}{\|q_r\|}.$$

To recover the translation, start from $q_d = \frac{1}{2}Tq_r$ and right-multiply by q_r^{-1} . Since q_r is unit, $q_r^{-1} = q_r^*$:

$$2q_d q_r^* = T(q_r q_r^*) = T \quad \implies \quad T = 2q_d q_r^*.$$

The vector part of the resulting pure quaternion T is the recovered translation $t = (T_x, T_y, T_z)$; its scalar component is zero up to floating-point error.

3. Why dual quaternions for VR blending?

Natural VR motions — a controller swinging through an arc, a shoulder-elbow-wrist chain articulating — combine rotation and translation along a single curved path, not two independent straight-line channels. Blending position and rotation separately (`Vector3.Lerp` + `Quaternion.Slerp`) forces the object along a flat chord between the two endpoints while the orientation turns independently, which is exactly the decoupling that produces "candy-wrapper" mesh collapse in skinning and visibly unnatural sliding motion for a single rigid object. A dual quaternion blends translation and rotation as one coupled 6-DOF quantity, so the interpolated pose follows the true spatial screw motion between the two poses (Chasles' theorem) instead of an arbitrary decoupled path.

Composition (used by `DualQuaternion.Multiply`)

$$\hat{q}_a \hat{q}_b = (q_{r,a} q_{r,b}) + \epsilon (q_{r,a} q_{d,b} + q_{d,a} q_{r,b}).$$

`Multiply(a, b)` applies b first, then a : $R = R_a R_b$ and $t = R_a t_b + t_a$, matching Unity's own quaternion/transform composition order.

Part B — Implementation (Unity C#)

`Assets/Scripts/DualQuaternion.cs` implements the struct exactly as specified:

- **Fields:** `Quaternion r` (real part), `Quaternion d` (dual part).
- `FromPose(rotation, translation)` — builds $\hat{q} = q_r + \epsilon(\frac{1}{2}Tq_r)$.
- `ToPose(out rotation, out translation)` — extracts $R = q_r / \|q_r\|$ and $t = 2q_d q_r^*$.
- `Multiply(a, b)` — rigid composition as derived above.
- `Slerp(a, b, u)` — dual quaternion linear blending (DLB): shortest-path sign alignment on the real part, `Quaternion.Slerp` on q_r , a *componentwise* `Mathf.Lerp` on q_d , then re-normalization.
- `Normalized()` / `Conjugate()` — enforce $\|q_r\| = 1$, $q_r \cdot q_d = 0$, and provide the dual quaternion conjugate.

Two implementation details that matter. Unity's `Quaternion` type has no operators for quaternion addition or scalar multiplication between two quaternions, so every dual-part operation is written out componentwise rather than relying on `Quaternion` arithmetic. Second, the dual part of the identity pose is the *zero* quaternion $(0, 0, 0, 0)$, not `Quaternion.identity` (which is $(0, 0, 0, 1)$) — using the latter would silently encode a phantom translation. Likewise, `Slerp` blends the dual part with `Mathf.Lerp` component-by-component rather than `Quaternion.Lerp`, which renormalizes its result to a unit quaternion and would corrupt the (generally non-unit) dual part.

Demo scene

`Assets/Scripts/Editor/SceneSetupHelper.cs` adds an editor menu item, **AM5011 → Build Full Demo Scene**, that constructs the scene programmatically (since a hand-authored `.unity` file cannot be produced outside the Unity Editor reliably):

- `PoseA` (red) and `PoseB` (green) as fixed pose anchors.
- `InterpolatedPose` (cyan), driven by `DualQuaternionPoseDemo.cs` using `DualQuaternion.Slerp`.
- `DecoupledPose` (magenta), driven by plain `Vector3.Lerp` and `Quaternion.Slerp` for the Part C comparison.
- A UI `Slider` (0–1) bound to the interpolation parameter u , plus a `TrailRenderer` on each pointer to visualize the swept path.

`Assets/Scripts/DualQuaternionMathTests.cs` is a Play-mode sanity check: it verifies the `FromPose/ToPose` roundtrip, that `Slerp` exactly recovers both endpoints at $u = 0, 1$, that the blended pose stays a valid unit dual quaternion at $u = 0.5$, and that `Multiply` composes two poses the same way Unity's own transform chaining does.

Part C — Mini-Experiment: Controller Interpolation

Setup. PoseA is placed at $(-2, 0, 0)$ m with identity rotation; PoseB is placed at $(2, 0, 0)$ m rotated 120° about $+Z$. Both methods interpolate between the identical pair of endpoints:

1. **Separate:** `Vector3.Lerp` on position, `Quaternion.Slerp` on rotation.
2. **Dual-quaternion Slerp (DLB):** `DualQuaternion.Slerp(dqA, dqB, u)`.

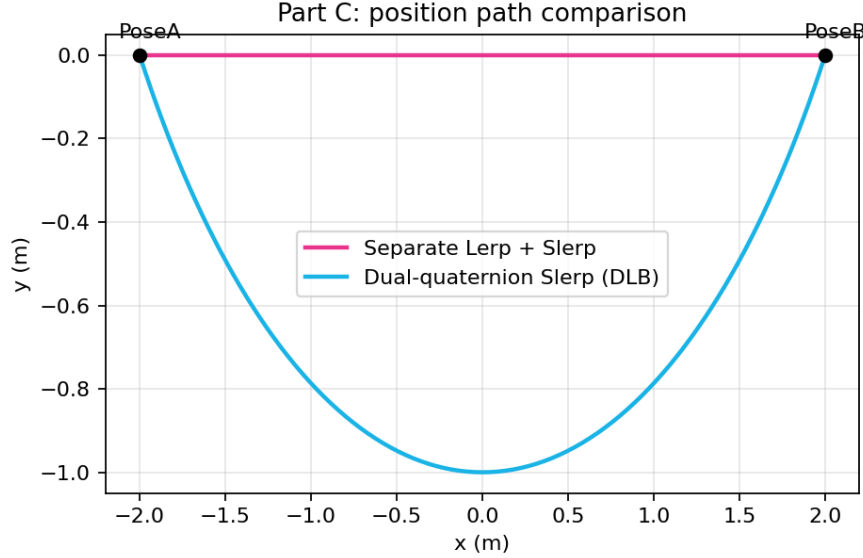


Figure 1: Computed position path for both methods, sampled at 101 values of $u \in [0, 1]$ (`tools/dualquat_sim.py`, a numeric replica of `DualQuaternion.cs`).

u	DLB x (m)	DLB y (m)	DLB angle	Separate angle
0.00	-2.000	0.000	0.00°	0.00°
0.25	-1.095	-0.742	30.00°	30.00°
0.50	0.000	-1.000	60.00°	60.00°
0.75	1.095	-0.742	90.00°	90.00°
1.00	2.000	0.000	120.00°	120.00°

Table 1: Position and orientation at five sample values of u .

Observations

- **Curved vs. straight path:** the separate method's position follows the straight chord $y = 0$ between the anchors; DLB's position follows a curved arc dipping to $y = -1.0$ m at $u = 0.5$. Coupling rotation into the translation term ($q_d = \frac{1}{2}Tq_r$) is exactly what bends the path.
- **Orientation profile is identical here:** both methods compute the rotation as `Quaternion.Slerp` of the same endpoint quaternions, so the angle-vs- u curve matches exactly in this experiment. The two methods diverge only in *where the translation goes*, not in the rotation timing — for a genuine screw motion (dual quaternion) this coupling is precisely what decoupled Lerp + Slerp cannot express.

- **Naturalness for VR:** the curved DLB path is consistent with a joint (shoulder, elbow, wrist) sweeping a controller through an arc; the straight-chord path of separate interpolation looks mechanical and unnatural for a rotating hand or tool, since it is only correct when translation and rotation are physically independent.
- **Rigidity is preserved either way:** because `Normalized()` is applied after every blend, DLB stays on the unit dual-quaternion manifold throughout $u \in [0, 1]$; there is no collapse or scale drift at the midpoint.
- **Caveat:** this simple DLB (linear blend + re-normalize) is an approximation to true constant-speed ScLERP; for large rotation angles or widely separated anchors the coupling is even more pronounced, which is the effect exploited for avatar-skinning artifact reduction (Kavan et al., 2008).

Deliverables

- `Assets/Scripts/DualQuaternion.cs`, `DualQuaternionPoseDemo.cs`, `Editor/SceneSetupHelper.cs`, `DualQuaternionMathTests.cs`.
- `tools/dualquat_sim.py` — numeric verification and Part C figure and table generator.
- This report (`report/main.tex`).